OOPS Concepts – Complete Notes

Object-Oriented Programming (OOP) concepts for beginners, interviews, and technical assessments

1. What is OOP?

Object-Oriented Programming (OOP) is a programming approach in which a program is designed using objects and classes. An object represents a real-world entity and contains data (state) and functions (behavior). OOP helps make programs modular, reusable, maintainable, and easier to extend.

Example: A Car can be treated as an object. Its data may include color and speed, while its behavior may include start(), stop(), and accelerate().

2. Class

A class is a blueprint or template used to create objects. It defines the properties (data) and methods (behavior) that its objects can have. A class itself is not normally a physical object; objects are instances created from the class.

Example in Java:

```
class Car {
    String color;
    void start() { }
}
```

3. Object

An object is an instance of a class. It has its own state and can perform the behaviors defined by its class. Objects are generally created using the **new** keyword in Java.

```
Car c1 = new Car();
```

Here, Car is the class and c1 is an object (instance) of Car.

4. The Four Main Pillars of OOP

Concept	Meaning
Encapsulation	Wrapping data and methods together and controlling access to the data.
Inheritance	Creating a new class from an existing class to reuse properties and methods.
Polymorphism	One interface/name can represent different behaviors.
Abstraction	Showing essential details while hiding unnecessary implementation details.

5. Encapsulation

Encapsulation means combining data and the methods that operate on that data inside a class and restricting direct access when necessary. In Java, encapsulation is commonly achieved using private fields and public getter/setter methods.

```
class Student {
    private int age;

    public void setAge(int age) {
        if (age >= 0) this.age = age;
```

```
    }

    public int getAge() {
        return age;
    }
}
```

Benefits: data protection, controlled access, validation, and easier maintenance.

6. Inheritance

Inheritance allows one class (child/subclass) to acquire accessible properties and methods from another class (parent/superclass). It promotes code reuse and represents an 'is-a' relationship.

```
class Animal {
    void eat() { System.out.println("Eating"); }
}

class Dog extends Animal {
    void bark() { System.out.println("Barking"); }
}
```

Dog can use both its own bark() method and the inherited eat() method.

Common types: single, multilevel, and hierarchical inheritance. Java does not support multiple inheritance of classes, but multiple inheritance of type can be achieved through interfaces.

7. Polymorphism

Polymorphism means 'many forms'. The same method name or interface can result in different behavior depending on the situation.

A. Compile-time polymorphism (Method Overloading)

Method overloading occurs when multiple methods in the same class have the same name but different parameter lists.

```
class Calculator {
    int add(int a, int b) { return a + b; }
    int add(int a, int b, int c) { return a + b + c; }
}
```

B. Runtime polymorphism (Method Overriding)

Method overriding occurs when a subclass provides its own implementation of a method already defined in the parent class. The method that executes can be determined at runtime based on the actual object.

```
class Animal { void sound() { System.out.println("Animal sound"); } }
class Dog extends Animal { void sound() { System.out.println("Bark"); } }

Animal a = new Dog();
a.sound(); // Bark
```

8. Abstraction

Abstraction means exposing only the essential functionality while hiding implementation details. In Java, abstraction can be achieved using abstract classes and interfaces.

```
abstract class Vehicle {
    abstract void start();
```

```
}

class Car extends Vehicle {
    void start() { System.out.println("Car starts"); }
}
```

The user of Vehicle needs to know that start() exists, but the implementation can differ for different vehicle types.

9. Constructor

A constructor is a special member of a class used to initialize objects. It has the same name as the class and does not have a return type. It is automatically called when an object is created.

```
class Student {
    String name;
    Student(String name) {
        this.name = name;
    }
}
```

Common forms include a no-argument constructor and a parameterized constructor. If no constructor is explicitly written, Java can provide a default constructor under appropriate conditions.

10. this Keyword

this refers to the current object. It is commonly used when a parameter name and an instance variable have the same name.

```
this.name = name;
```

11. super Keyword

super refers to the immediate parent-class portion of the current object. It can be used to access a parent class member or invoke a parent constructor.

```
super.display();
```

12. Access Modifiers

Modifier	Access
private	Within the same class
default (no modifier)	Within the same package
protected	Same package and subclasses (with Java's access rules)
public	Accessible wherever the class/member is accessible

13. Interface

An interface defines a contract that implementing classes agree to follow. It is useful for abstraction and for defining common behavior across otherwise unrelated classes. A class uses the **implements** keyword to implement an interface.

```
interface Payment {
    void pay();
}

class UPI implements Payment {
    public void pay() { System.out.println("Paid using UPI"); }
}
```

14. Abstract Class vs Interface

Abstract Class	Interface
Declared using abstract class	Declared using interface
Can have instance variables	Fields are constants (public static final by default)
Can have constructors	Cannot have constructors
A class extends one class	A class can implement multiple interfaces
Can contain abstract and concrete methods	Can contain abstract methods and, in modern Java, default/static method

15. Association, Aggregation, and Composition

Association: a general relationship between two independent objects. Example: Teacher teaches Student.
Aggregation: a weak 'has-a' relationship where the contained object can exist independently. Example: Department has Teachers.
Composition: a strong 'has-a' relationship where the contained object's lifecycle is tied to the owner. Example: House has Rooms.

16. Method Overloading vs Method Overriding

Overloading	Overriding
Usually compile-time polymorphism	Runtime polymorphism
Same method name, different parameter list	Same method signature in parent and child
Can occur within the same class	Requires inheritance
Return type alone cannot distinguish overloaded methods	Return type must be compatible with the overridden method

17. Important OOP Interview Questions

1. What is OOP and why is it used?

2. What is the difference between a class and an object?

3. Explain the four pillars of OOP.

4. What is encapsulation? Why are fields often private?

5. What is inheritance? What types of inheritance are common?

6. What is polymorphism? Explain overloading and overriding.

7. What is abstraction? How is it achieved in Java?

8. What is a constructor? Can a constructor be inherited?

9. What is the difference between this and super?

10. What is the difference between an abstract class and an interface?

11. What is method overloading?

12. What is method overriding?

13. What is the difference between == and equals() in Java?

14. What is association, aggregation, and composition?

15. Why is OOP useful for large software projects?

18. Quick Revision

Class → Blueprint
Object → Instance of a class
Encapsulation → Bundle data + methods and control access
Inheritance → Reuse through parent-child relationship
Polymorphism → One name/interface, multiple behaviors
Abstraction → Hide implementation, expose essentials
Constructor → Initializes an object
this → Current object
super → Parent-class reference